MODUL PRAKTIKUM 5
PEMROGRAMAN BERORIENTASI OBJEK LANJUT

5.1. Topik Bahasan

Modul ini dilaksanakan pada pertemuan ke-5 sesuai Rencana Pembelajaran Semester mata kuliah Pemrograman IV dan membebani CPMK-02, dengan cakupan topik sebagai berikut:

- Pewarisan kelas pada moda transportasi pengiriman.
- Penggantian metode (method overriding) pada perhitungan tarif.
- Kelas abstrak sebagai kerangka layanan pengiriman.
- Polimorfisme melalui pemanggilan metode pada tipe induk.
- Penggunaan *mixin* untuk perilaku bersama antar kelas.

5.2. Capaian Pembelajaran

Mahasiswa mampu menerapkan pewarisan, polimorfisme, dan abstraksi pada program aplikasi logistik (CPMK-02).

5.3. Indikator Ketercapaian Pembelajaran

Indikator penilaian pertemuan ke-5 menurut Rencana Pembelajaran Semester adalah:

- Ketepatan penerapan pewarisan dan enkapsulasi
- Ketepatan penerapan polimorfisme, kelas abstrak, dan mixin

Indikator tersebut dinyatakan tercapai apabila bukti berikut terlihat pada hasil praktikum:

- Hierarki kelas berjalan sesuai rancangan dan tidak terdapat pengulangan kode antar turunan.
- Setiap kelas turunan menghasilkan perhitungan tarif yang berbeda sesuai karakteristik modanya.
- Kelas abstrak dan mixin digunakan pada tempat yang tepat serta disertai alasan yang dapat dijelaskan.
- Pemanggilan metode melalui tipe induk menghasilkan perilaku kelas turunan yang benar.

5.4. Teori Pendukung

Pewarisan

Pewarisan (inheritance) memungkinkan sebuah kelas mengambil alih atribut dan metode kelas lain melalui *keyword* `extends`. Kelas yang mewariskan disebut kelas induk (superclass) dan kelas yang mewarisi disebut kelas turunan (subclass). Tujuan

utamanya adalah menghindari penulisan ulang kode yang sama pada beberapa kelas yang memiliki kesamaan sifat.

Pada kasus logistik, pengiriman melalui truk, kereta api, dan kapal memiliki kesamaan: semuanya memiliki kode layanan, jarak tempuh, dan berat muatan. Perbedaannya terletak pada cara menghitung tarif dan estimasi waktu. Kesamaan tersebut diletakkan pada kelas induk, sedangkan perbedaannya diwujudkan pada tiap kelas turunan.

Penggantian Metode dan Keyword super

Kelas turunan dapat menuliskan ulang metode milik kelas induk. Penulisan ulang ini ditandai dengan anotasi `@override` yang membantu *compiler* memeriksa bahwa nama dan tanda tangan metode benar-benar cocok. Apabila implementasi kelas induk masih dibutuhkan, keyword `super` dipakai untuk memanggilnya, baik pada metode maupun pada konstruktor.

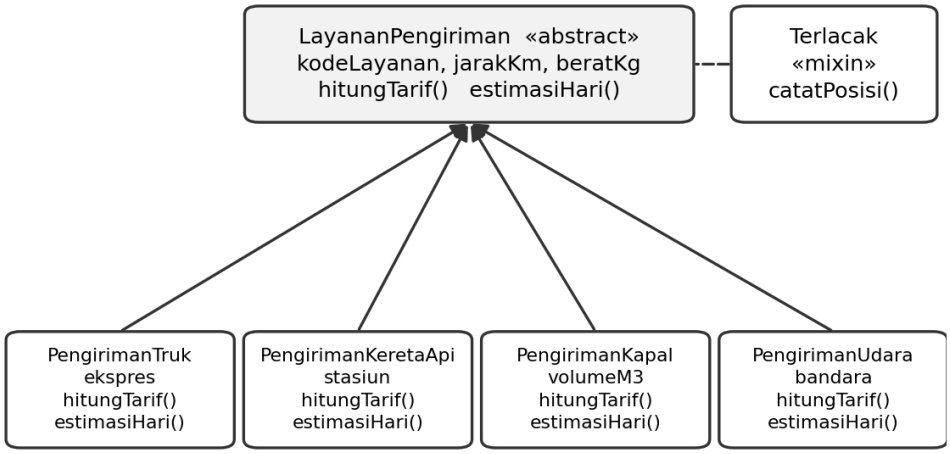
Kelas Abstrak

Kelas abstrak adalah kelas yang tidak dapat dibentuk menjadi objek dan berfungsi sebagai kerangka kontrak bagi kelas turunannya. Kelas abstrak boleh memuat metode abstrak, yaitu metode tanpa badan, yang wajib diisi oleh setiap turunannya. Dengan demikian kelas abstrak menjamin bahwa seluruh moda pengiriman pasti memiliki metode `hitungTarif` dan `estimasiHari`, tanpa menentukan bagaimana keduanya dihitung.

Kelas abstrak juga boleh memuat metode biasa yang sudah berisi implementasi umum, misalnya metode untuk mencetak ringkasan layanan. Turunan cukup mewarisinya tanpa perlu menulis ulang.

Polimorfisme

Polimorfisme berarti satu antarmuka dapat berperilaku berbeda bergantung pada objek yang sebenarnya. Sebuah daftar bertipe `List<LayananPengiriman>` dapat berisi objek truk, kereta api, maupun kapal. Ketika metode `hitungTarif()` dipanggil untuk tiap elemen, Dart secara otomatis menjalankan implementasi yang sesuai dengan kelas objek tersebut. Karena itu penambahan moda baru tidak memaksa perubahan pada kode yang sudah ada.



Gambar 5. Hierarki kelas layanan pengiriman beserta mixin Terlacak

Mixin

Dart tidak mendukung pewarisan berganda, namun menyediakan mixin untuk berbagi perilaku ke banyak kelas yang tidak berada pada satu jalur pewarisan. Mixin dideklarasikan dengan keyword `mixin` dan dipasang melalui keyword `with`. Mixin cocok untuk perilaku yang bersifat tambahan dan lintas jenis, misalnya kemampuan melacak posisi atau kemampuan mencatat riwayat perubahan.

Pewarisan dipakai ketika terdapat hubungan "adalah sebuah", misalnya truk adalah sebuah layanan pengiriman. Mixin dipakai ketika terdapat hubungan "memiliki kemampuan", misalnya truk memiliki kemampuan dilacak.

Antarmuka Implisit

Setiap kelas pada Dart secara otomatis mendefinisikan sebuah antarmuka. Keyword `implements` memaksa kelas menuliskan ulang seluruh anggota kelas yang diacu tanpa mewarisi implementasinya. Penggunaannya tepat ketika hanya bentuk kontrak yang dibutuhkan, bukan kode bersamanya.

Tabel 6. Perbandingan mekanisme penggunaan ulang kode pada bahasa Dart

Mekanisme	Keyword	Mewarisi implementasi	Dipakai ketika
Pewarisan	`extends`	Ya, satu kelas induk	Terdapat hubungan "adalah sebuah"
Mixin	`with`	Ya, boleh lebih dari satu	Terdapat hubungan "memiliki kemampuan"
Antarmuka implisit	`implements`	Tidak, hanya bentuk kontrak	Hanya kontrak yang dibutuhkan

5.5. Alat Pendukung

Perangkat keras, perangkat lunak, dan bahan yang diperlukan pada modul ini adalah:

- Flutter SDK dan Dart SDK.
- Visual Studio Code beserta ekstensi Flutter dan Dart.
- Terminal PowerShell.
- Perangkat lunak pembuat diagram kelas (draw.io).
- Repositori GitHub pribadi praktikan (satu repositori untuk seluruh modul praktikum).

5.6. Langkah-langkah Praktikum

A. Penyiapan

1. Buka proyek `silog_app` melalui menu File > Open Folder pada Visual Studio Code, lalu buka terminal terintegrasi melalui menu Terminal > New Terminal. Buat cabang kerja modul ini pada terminal dengan perintah `git checkout -b praktikum/modul-05`, kemudian pastikan cabang aktif melalui `git branch`.
2. Buat direktori `lib/layanan` melalui panel Explorer: klik kanan direktori `lib`, pilih New Folder, lalu ketik `layanan`.
3. Gambarkan hierarki kelas layanan pengiriman melalui draw.io pada alamat `https://app.diagrams.net` sebelum menulis kode, lalu simpan melalui menu File > Export as > PNG.

B. Kelas Abstrak dan Mixin

1. Buat berkas `lib/layanan/layanan_pengiriman.dart` melalui panel Explorer: klik kanan direktori `lib/layanan`, pilih New File, lalu ketik nama berkas `layanan_pengiriman.dart`. Ketik kode berikut, kemudian simpan melalui `Ctrl+S`.

Kode Program 5.1 Kelas abstrak dan mixin

```dart
mixin Terlacak {
  final List<String> _riwayat = [];

  List<String> get riwayat => List.unmodifiable(_riwayat);

  void catatPosisi(String lokasi) {
    final waktu = DateTime.now().toIso8601String().substring(11, 16);
    _riwayat.add('[$waktu] $lokasi');
  }

  String posisiTerakhir() =>
      _riwayat.isEmpty ? 'Belum ada data posisi' : _riwayat.last;
}

abstract class LayananPengiriman {
  final String kodeLayanan;
  final double jarakKm;
  final double beratKg;

  LayananPengiriman({
    required this.kodeLayanan,
    required this.jarakKm,
    required this.beratKg,
  });

  /// Wajib diisi oleh setiap moda pengiriman.
```

```
  double hitungTarif();

  /// Wajib diisi oleh setiap moda pengiriman.
  int estimasiHari();

  /// Implementasi umum yang diwarisi seluruh turunan.
  String ringkasan() {
    final tarif = hitungTarif().toStringAsFixed(0);
    return '$kodeLayanan | ${jarakKm.toStringAsFixed(0)} km | '
        '${beratKg.toStringAsFixed(1)} kg | Rp$tarif | ${estimasiHari()} hari';
  }
}
```

C. Kelas Turunan Moda Transportasi

1. Buat berkas `lib/layanan/moda.dart` melalui panel Explorer: klik kanan direktori `lib/layanan`, pilih New File, lalu ketik nama berkas `moda.dart`. Ketik kode berikut, kemudian simpan melalui `Ctrl+S`.

Kode Program 5.2 Tiga moda pengiriman sebagai kelas turunan

```
import 'layanan_pengiriman.dart';

class PengirimanTruk extends LayananPengiriman with Terlacak {
  final bool ekspres;

  PengirimanTruk({
    required super.kodeLayanan,
    required super.jarakKm,
    required super.beratKg,
    this.ekspres = false,
  });

  @override
  double hitungTarif() {
    const tarifDasarPerKm = 4500;
    const tarifPerKg = 1200;
    final tarif = (jarakKm * tarifDasarPerKm) + (beratKg * tarifPerKg);
    return ekspres ? tarif * 1.5 : tarif;
  }

  @override
  int estimasiHari() {
    final hari = (jarakKm / 450).ceil();
    return ekspres ? (hari / 2).ceil() : hari;
  }
}

class PengirimanKeretaApi extends LayananPengiriman with Terlacak {
  final String stasiunAsal;
  final String stasiunTujuan;

  PengirimanKeretaApi({
    required super.kodeLayanan,
    required super.jarakKm,
    required super.beratKg,
    required this.stasiunAsal,
    required this.stasiunTujuan,
  });

  @override
  double hitungTarif() {
    const tarifPerKmKg = 95;
```

```
    const biayaBongkarMuat = 250000;
    return (jarakKm * beratKg * tarifPerKmKg / 100) + biayaBongkarMuat;
  }

  @override
  int estimasiHari() => (jarakKm / 700).ceil() + 1;

  @override
  String ringkasan() => '${super.ringkasan()} | $stasiunAsal-$stasiunTujuan';
}

class PengirimanKapal extends LayananPengiriman with Terlacak {
  final double volumeM3;
  final bool petiKemas;

  PengirimanKapal({
    required super.kodeLayanan,
    required super.jarakKm,
    required super.beratKg,
    required this.volumeM3,
    this.petiKemas = true,
  });

  @override
  double hitungTarif() {
    const tarifPerM3 = 850000;
    const biayaPelabuhan = 1750000;
    final tarif = (volumeM3 * tarifPerM3) + biayaPelabuhan;
    return petiKemas ? tarif : tarif * 1.2;
  }

  @override
  int estimasiHari() => (jarakKm / 320).ceil() + 3;
}
```

D. Pengujian Polimorfisme

1. Buat berkas `latihan/modul05_uji.dart` melalui panel Explorer: klik kanan direktori `latihan`, pilih New File, lalu ketik nama berkas `modul05_uji.dart`. Ketik kode berikut, kemudian simpan melalui `Ctrl+S`.

Kode Program 5.3 Pemanggilan metode melalui tipe induk

```
import '../lib/layanan/layanan_pengiriman.dart';
import '../lib/layanan/moda.dart';

void main() {
  final List<LayananPengiriman> daftarLayanan = [
    PengirimanTruk(
        kodeLayanan: 'TRK-REG', jarakKm: 780, beratKg: 1200),
    PengirimanTruk(
        kodeLayanan: 'TRK-EXP', jarakKm: 780, beratKg: 1200, ekspres: true),
    PengirimanKeretaApi(
      kodeLayanan: 'KA-CARGO',
      jarakKm: 780,
      beratKg: 1200,
      stasiunAsal: 'Kiaracondong',
      stasiunTujuan: 'Pasar Turi',
    ),
    PengirimanKapal(
      kodeLayanan: 'KPL-PETI',
      jarakKm: 1450,
      beratKg: 1200,
```

```
      volumeM3: 8.5,
    ),
  ];

  print('=== Perbandingan Layanan Pengiriman ===');
  for (final layanan in daftarLayanan) {
    print(layanan.ringkasan());
  }

  daftarLayanan.sort((a, b) => a.hitungTarif().compareTo(b.hitungTarif()));
  print('\nLayanan termurah : ${daftarLayanan.first.kodeLayanan}');
  print('Layanan tercepat : '
      '${daftarLayanan.reduce((a, b) => a.estimasiHari() <= b.estimasiHari() ? a :
b).kodeLayanan}');

  // Pemanfaatan mixin Terlacak
  final truk = daftarLayanan
      .whereType<PengirimanTruk>()
      .first;
  truk.catatPosisi('Gudang Utama Bandung');
  truk.catatPosisi('Rest Area KM 207');
  truk.catatPosisi('Gudang Transit Semarang');
  print('\nRiwayat posisi ${truk.kodeLayanan}:');
  for (final r in truk.riwayat) {
    print('  $r');
  }
  print('Posisi terakhir: ${truk.posisiTerakhir()}');
}
```

2. Jalankan program melalui `dart run latihan/modul05_uji.dart`.
3. Bandingkan tarif keempat layanan dan jelaskan penyebab perbedaannya berdasarkan rumus pada masing-masing kelas.
4. Cobalah membentuk objek `LayananPengiriman(...)` secara langsung, amati pesan *error* dari compiler, lalu jelaskan sifat kelas abstrak berdasarkan pesan tersebut.

E. Latihan Penambahan Moda Baru

1. Tambahkan kelas `PengirimanUdara` yang mewarisi `LayananPengiriman` dan memakai mixin `Terlacak`.
2. Tetapkan rumus tarif yang memperhitungkan berat volumetrik dari Modul 2 dan biaya bandara tetap.
3. Tambahkan objek moda baru tersebut ke dalam `daftarLayanan`, simpan melalui `Ctrl+S`, jalankan ulang melalui `dart run latihan/modul05_uji.dart`, dan pastikan tidak ada baris kode lain yang perlu diubah. Jelaskan pada jurnal mengapa hal ini dimungkinkan.

F. Tugas Praktikum

1. Buat mixin baru bernama `Terasuransi` yang menyediakan metode `hitungPremi(double nilaiBarang)` dan atribut persentase premi.

2. Pasang mixin tersebut pada `PengirimanKapal` dan `PengirimanUdara` saja, lalu tunjukkan melalui program bahwa `PengirimanTruk` tidak memiliki metode tersebut.
3. Lampirkan diagram hierarki kelas akhir pada jurnal beserta penjelasan pemilihan pewarisan dan mixin.

G. Luaran yang Dinilai

Bentuk penilaian modul ini adalah tugas praktikum dan kuis dengan kriteria kebenaran hierarki kelas dan ketepatan jawaban kuis. jurnal praktikum individu diunggah ke lms ulbi paling lambat 5x24 jam setelah sesi praktikum. Agar kriteria tersebut dapat dinilai, praktikan menyerahkan luaran berikut:

- Diagram hierarki kelas layanan pengiriman, memperlihatkan kelas abstrak, kelas turunan, dan mixin yang dipasang.
- Berkas `layanan_pengiriman.dart` dan `moda.dart` termasuk moda tambahan hasil tugas praktikum.
- Tangkapan layar *output* perbandingan tarif seluruh moda beserta penjelasan penyebab perbedaannya.
- Bukti bahwa penambahan moda baru tidak mengubah kode pemanggil, disertai penjelasan singkat mengenai polimorfisme.
- Lembar jawaban kuis.

5.7. Panduan Pengumpulan Jurnal Hasil Kegiatan Praktikum

Tugas praktikum modul ini bersifat individu. Setiap praktikan menyusun dan mengunggah sendiri jurnalnya ke LMS ULBI paling lambat 5 x 24 jam setelah sesi praktikum berakhir, berupa berkas PDF bernama `NIM_Nama_Modul-XX.pdf`, disertai tautan repositori GitHub pribadi pada cabang `praktikum/modul-XX`.

Sistematika jurnal:

1. Identitas: nama, NIM, kelas, nomor dan judul modul, serta tanggal praktikum.
2. Tujuan praktikum.
3. Alat dan bahan yang digunakan beserta versinya.
4. Langkah kerja, disertai tangkapan layar tiap tahap penting.
5. Kode program beserta penjelasan singkatnya.
6. Hasil eksekusi dan analisisnya, termasuk error yang ditemui beserta penyelesaiannya.
7. Jawaban seluruh butir Tugas Praktikum.
8. Kesimpulan dan kendala.
9. Tautan repositori beserta tangkapan layar riwayat *commit*.

Tangkapan layar wajib memperlihatkan jendela *browser* atau emulator secara utuh beserta kode programnya, dan memuat nama serta NIM praktikan. Praktikan yang tidak

mengumpulkan jurnal memperoleh nilai nol; keterlambatan dan ketidaksesuaian hasil lainnya dikenai pengurangan nilai sesuai kebijakan dosen pengampu.

Seluruh butir pada sub bab Luaran yang Dinilai wajib dilampirkan pada jurnal.

5.8. Kuis

Kuis dilaksanakan pada akhir sesi praktikum selama 20 menit, setelah seluruh langkah praktikum selesai dikerjakan. Kuis menilai pemahaman konsep, bukan kecepatan mengetik kode, sehingga dikerjakan tanpa membuka berkas program.

Bentuk dan cakupan soal:

- Delapan soal pilihan ganda mengenai pewarisan, enkapsulasi, polimorfisme, kelas abstrak, dan mixin.
- Dua soal penelusuran potongan kode: menentukan metode mana yang dijalankan ketika sebuah metode dipanggil melalui tipe induk.
- Satu soal analisis: menentukan kapan sebuah perilaku sebaiknya diwujudkan sebagai pewarisan dan kapan sebagai mixin, disertai alasan.

Nilai kuis menjadi bagian dari komponen evaluasi Quiz sebesar 5% dari nilai akhir mata kuliah, terpisah dari nilai tugas praktikum.

5.9. Hasil Capaian Pembelajaran Praktikum

Bentuk dan kriteria penilaian mengikuti Rencana Pembelajaran Semester pertemuan ke-5: Tugas praktikum dan kuis: kebenaran hierarki kelas dan ketepatan jawaban kuis. Jurnal praktikum individu diunggah ke LMS ULBI paling lambat 5x24 jam setelah sesi praktikum.

Tugas praktikum modul ini bersifat individu dan dinilai dari jurnal praktikum masing-masing praktikan.

Bobotnya 2% dari nilai akhir mata kuliah pada komponen evaluasi Praktikum (Aktivitas Partisipatif) sebesar 15% dari nilai akhir, Quiz (Kognitif/Pengetahuan) sebesar 5% dari nilai akhir, diukur dari kelengkapan dan mutu luaran pada sub bab Luaran yang Dinilai terhadap indikator penilaian modul ini.